Technical Write-up: GoComet Nova

1. System Architecture

The following diagram illustrates the data flow, agent boundaries, and state management within the Nova platform.

Key Architectural Decisions:

- **State Lives in the Database:** LangGraph's state is backed by a SQLite checkpointer. This means the graph state is fully serialized and saved at every node transition, ensuring durability.
- **Sharp Agent Boundaries:** LLMs handle extraction, validation, and drafting. The Validator Agent is a true Tool-Calling Agent that uses Anthropic's parallel tool calling to delegate complex math and string logic to deterministic Python functions, completely eliminating hallucination risk for numeric checks while maintaining the intelligence of an LLM.

---

2. The Three Nastiest Failure Modes

During testing and development, I encountered several complex failure modes that standard prompts couldn't handle.

1. The "Clean" Shipment False Failure

- **The Reality:** Our "clean" test shipment (`shipment_1`) was failing validation. The customer rules demanded an `invoice_number`, but the agent couldn't find one on the Bill of Lading (BOL).
- **The Issue:** BOLs generally *do not have* invoice numbers; they belong on the Commercial Invoice. A naive validator applies all rules to all documents.
- **The Fix:** Implemented a **Document-Type Awareness Layer**. The Validator now filters rules based on the document type, skipping `required` checks for fields that don't naturally belong on that specific document, preventing false discrepancies.

2. Cross-Document Formatting Discrepancies

- **The Reality:** The Gross Weight on the BOL was listed as `4,250 KG`, but on the Packing List, it was written as `4.250 MT`.
- **The Issue:** A simple string equality check or basic LLM comparison fails here. A human knows these are identical, but the machine flags a discrepancy, lowering the Straight-Through Processing (STP) rate.
- **The Fix:** Built a robust `_extract_numeric` parser in the Validator that intelligently isolates floats and units, normalizes them to a common base (e.g., converting Metric Tons to Kilograms), and applies a customer-defined percentage tolerance (e.g., $\pm 0.5\%$) before comparing.

3. Mid-Pipeline State Loss (The Silent Crash)

- **The Reality:** If the FastAPI server restarted while a large PDF was being extracted, the pipeline died. The CG operator was left with a shipment perpetually stuck in "Processing".
- **The Issue:** LangGraph uses an in-memory `MemorySaver` by default.
- **The Fix:** Implemented a custom `SQLiteBackedMemorySaver`. Because LangGraph allows custom checkpointers, we intercept the blobs at every node and write them to `nova.db`. If the server crashes, a recovery script can re-instantiate the graph using the `thread_id` and resume exactly from the failed node.

---

4. Observability: Tracing a Shipment at Scale

If Nova is running in production for 50 customers processing thousands of documents, simple console logs are useless. Here is how observability is structured:

Tracing a Single Shipment

1. **Unique Trace ID:** The moment an email arrives, a unique `shipment_id` and `thread_id` are generated. This ID propagates through Kafka, FastAPI, LangGraph, and the UI.
2. **LLM Observability (Langfuse / OpenTelemetry):** We wrap every Claude API call in a Langfuse tracer. If a specific extraction fails, we can pull up the trace to see the exact base64 image sent, the prompt, the latency, and the exact token usage for that specific hop.
3. **State Emissions (SSE):** The backend emits Server-Sent Events (SSE) at every graph node boundary (`extracting`, `validated`, `decided`). The UI subscribes to this stream, allowing operators to see exactly where a shipment is in the pipeline in real-time.

The Production Dashboard

An FDE or Engineering Manager looking at the Nova dashboard would see:

- **Straight-Through Processing (STP) Rate:** The golden metric.
- **Average Confidence Score by Supplier:** Highlights which suppliers send low-quality scans.
- **Error Rate by Rule:** Shows which validation rules fail most often (indicating a need for supplier retraining or rule loosening).
- **P95 Pipeline Latency:** To monitor if the vision model is degrading in speed.

---

5. Cost Analysis & Control

Using **Claude 3.5 Sonnet**, the unit economics are highly favorable for large-scale operations.

Back-of-the-Envelope Cost (Per Document)

- **Claude 3.5 Sonnet Pricing:** ~$3.00 / 1M input tokens | ~$15.00 / 1M output tokens.
- **Usage:** A standard 2-page PDF (converted to images) consumes roughly 3,000 input tokens and 500 output tokens.

- **Calculation:**
    - Input: (3,000 / 1,000,000) * $3.00 = `$0.009`
    - Output: (500 / 1,000,000) * $15.00 = `$0.0075`
- **Total: ~$0.0165 per document.**
- A 4-document shipment costs roughly **~$0.066** to process end-to-end.

Where it Blows Up & How We Control It

1. **Infinite Agent Loops:** If the LLM repeatedly outputs invalid JSON, LangGraph could loop infinitely, racking up costs.
    - *Control:* Hard `recursion_limit` set in LangGraph. After 3 failed retries, it throws a `MaxRetriesExceeded` exception and routes to human review.
2. **Massive PDFs:** A supplier accidentally attaches a 200-page product catalog instead of a 1-page invoice. Sending 200 pages as images to the Vision API will spike costs instantly.
    - *Control:* Pre-processing checks. If a PDF exceeds 10 pages, it is rejected by the trigger and flagged for manual triage. Furthermore, we compress images to a maximum dimension (e.g., 1024px) before encoding to base64.

---

6. Latency Analysis

Where is the Slowest Hop?

The **Extractor Agent** is the absolute bottleneck.

- **Extraction:** Processing 4 heavy PDFs via the Vision API takes **~4-6 seconds** (even when parallelized, due to network I/O and Anthropic rate limits).
- **Validation:** Hybrid. Pure Python rules (regex, math) take $<$ **0.05 seconds**, while non-deterministic semantic reasoning takes **~1-2 seconds**.
- **Routing/Drafting:** LLM generation of a short text email. Takes **~1.5 - 2 seconds**.

How to Fix It

To shave seconds off the Extractor hop:

1. **Text-Layer Sniffing:** Before sending images to the Vision model, check if the PDF has a clean, extractable text layer (via PyMuPDF). If it does, we can pass text to the LLM instead of base64 images. Text processes significantly faster and consumes fewer tokens.
2. **Aggressive Async:** Ensure that the LangChain/Anthropic API calls are utilizing `asyncio.gather` so that all 4 documents in a shipment are extracted concurrently, reducing the latency to the speed of the single slowest document.

---

7. What I'd Do Differently With a Week

If I had a full week instead of a DAW assignment, I would elevate this from a robust POC to

enterprise-grade infrastructure:

1. **Migrate to ClickHouse & Postgres:** SQLite is insufficient for concurrent LangGraph pipeline executions. I would use Postgres for operational state and relationships (Customers, Rules, Shipments) and ClickHouse for the analytical storage (observability logs, extracted fields, confidence scores) as defined in the Nova architecture.
2. **Vector Database (Weaviate) for Rules:** Managing JSON rule files per customer does not scale. I would ingest customer SOPs into Weaviate, allowing the Validator to perform semantic searches to dynamically fetch rules (e.g., finding the specific demurrage clause for a given port).
3. **Real Event-Driven Trigger (Kafka / Inbox):** I would build a true email ingester (using AWS SES inbound parsing or Microsoft Graph API) that drops an event into a Kafka topic, which natively triggers the LangGraph worker pool.
4. **OpenFGA Integration:** Implement relationship-based access control (Zanzibar model) so that a CG operator can only view and validate shipments for their assigned tenants.